

Integrity Constraints

Integrity constraints ensure that changes made to the database by authorized users do not result in a loss of data consistency. Thus, integrity constraints guard against accidental damage to the database.

Examples of integrity constraints are:

- An instructor name cannot be *null*.
- No two instructors can have the same instructor ID.
- Every department name in the *course* relation must have a matching department name in the *department* relation.
- The budget of a department must be greater than \$0.00.

Screen clipping taken: 04-06-2026 15:23

4.4.1 Constraints on a Single Relation

Screen clipping taken: 04-06-2026 15:39

- **not null**
- **unique**
- **check(<predicate>)**

Screen clipping taken: 04-06-2026 15:39

```
name  varchar(20) not null
budget numeric(12,2) not null
```

The **not null** specification prohibits the insertion of a null value for the attribute. Any database modification that would cause a null to be inserted in an attribute declared to be **not null** generates an error diagnostic.

Screen clipping taken: 04-06-2026 15:43

4.4.3 Unique Constraint

SQL also supports an integrity constraint:

```
unique (Aj1, Aj2, ..., Ajm)
```

The **unique** specification says that attributes $A_{j_1}, A_{j_2}, \dots, A_{j_m}$ form a candidate key; that is, no two tuples in the relation can be equal on all the listed attributes. However, candidate key attributes are permitted to be *null* unless they have explicitly been declared to be **not null**. Recall that a null value does not equal any other value. (The treatment of nulls here is the same as that of the **unique** construct defined in Section 3.8.4.)

Screen clipping taken: 04-06-2026 15:44

4.4.4 The check Clause

When applied to a relation declaration, the clause **check**(*P*) specifies a predicate *P* that must be satisfied by every tuple in a relation.

A common use of the **check** clause is to ensure that attribute values satisfy specified conditions, in effect creating a powerful type system. For instance, a clause **check** (*budget* > 0) in the **create table** command for relation *department* would ensure that the value of **budget** is nonnegative.

As another example, consider the following:

```
create table section
(course_id    varchar (8),
 sec_id      varchar (8),
 semester    varchar (6),
 year        numeric (4,0),
 building     varchar (15),
 room_number varchar (7),
 time_slot_id varchar (4),
 primary key (course_id, sec_id, semester, year),
 check (semester in ('Fall', 'Winter', 'Spring', 'Summer')));
```

Screen clipping taken: 04-06-2026 15:45

An **assertion** is a predicate expressing a condition that we wish the database always to satisfy. Domain constraints and referential-integrity constraints are special forms of assertions. We have paid substantial attention to these forms of assertions because they are easily tested and apply to a wide range of database applications. However, there are many constraints that we cannot express by using only these special forms. Two examples of such constraints are:

- For each tuple in the *student* relation, the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully.
- An instructor cannot teach in two different classrooms in a semester in the same time slot.¹

An assertion in SQL takes the form:

```
create assertion <assertion-name> check <predicate>;
```

Screen clipping taken: 04-06-2026 15:48

SQL Data Types and Schemas

Screen clipping taken: 04-06-2026 15:48

4.5.1 Date and Time Types in SQL

Screen clipping taken: 04-06-2026 15:49

In addition to the basic data types we introduced in Section 3.2, the SQL standard supports several data types relating to dates and times:

- **date**: A calendar date containing a (four-digit) year, month, and day of the month.
- **time**: The time of day, in hours, minutes, and seconds. A variant, **time(p)**, can be used to specify the number of fractional digits for seconds (the default being 0). It is also possible to store time-zone information along with the time by specifying **time with timezone**.
- **timestamp**: A combination of **date** and **time**. A variant, **timestamp(p)**, can be used to specify the number of fractional digits for seconds (the default here being 6). Time-zone information is also stored if **with timezone** is specified.

Date and time values can be specified like this:

```
date '2001-04-25'
time '09:30:00'
timestamp '2001-04-25 10:29:01.45'
```

Dates must be specified in the format year followed by month followed by day, as shown. The seconds field of **time** or **timestamp** can have a fractional part, as in the timestamp above.

Screen clipping taken: 04-06-2026 15:51

SQL defines several functions to get the current date and time. For example, **current_date** returns the current date, **current_time** returns the current time (with time zone), and **localtime** returns the current local time (without time zone). Timestamps (date plus time) are returned by **current_timestamp** (with time zone) and **localtimestamp** (local date and time without time zone).

Screen clipping taken: 04-06-2026 15:53

4.5.2 Default Values

SQL allows a default value to be specified for an attribute as illustrated by the following **create table** statement:

```
create table student
(ID          varchar (5),
name        varchar (20) not null,
dept_name   varchar (20),
tot_cred    numeric (3,0) default 0,
primary key (ID));
```

The default value of the *tot_cred* attribute is declared to be 0. As a result, when a tuple is inserted into the *student* relation, if no value is provided for the *tot_cred* attribute, its value is set to 0. The following insert statement illustrates how an insertion can omit the value for the *tot_cred* attribute.

```
insert into student(ID, name, dept_name)
values ('12789', 'Newman', 'Comp. Sci.');
```

Screen clipping taken: 04-06-2026 15:54

4.5.3 Index Creation

Screen clipping taken: 04-06-2026 16:00

An **index** on an attribute of a relation is a data structure that allows the database system to find those tuples in the relation that have a specified value for that attribute efficiently, without scanning through all the tuples of the relation.

```
create index studentID_index on student(ID);
```

4.5.4 Large-Object Types

Many current-generation database applications need to store attributes that can be large (of the order of many kilobytes), such as a photograph, or very large (of the order of many megabytes or even gigabytes), such as a high-resolution medical image or video clip. SQL therefore provides large-object data types for character data (**clob**) and binary data (**blob**). The letters “lob” in these data types stand for “Large Object.” For example, we may declare attributes

```
book_review clob(10KB)  
image blob(10MB)  
movie blob(2GB)
```

4.5.5 User-Defined Types

SQL supports two forms of user-defined data types. The first form, which we cover here, is called **distinct types**. The other form, called **structured data types**, allows the creation of complex data types with nested record structures, arrays,

and multisets. We do not cover structured data types

4.5.6 Create Table Extensions

Applications often require creation of tables that have the same schema as an existing table. SQL provides a **create table like** extension to support this task:

```
create table temp_instructor like instructor;
```

The above statement creates a new table *temp_instructor* that has the same schema as *instructor*.

When writing a complex query, it is often useful to store the result of a query as a new table; the table is usually temporary. Two statements are required, one to create the table (with appropriate columns) and the second to insert the query result into the table. SQL:2003 provides a simpler technique to create a table containing the results of a query. For example the following statement creates a table *t1* containing the results of a query.

```
create table t1 as  
  (select *  
   from instructor  
   where dept_name= 'Music')  
with data;
```

Authorization

We may assign a user several forms of authorizations on parts of the database. Authorizations on data include:

- Authorization to read data.
- Authorization to insert new data.
- Authorization to update data.
- Authorization to delete data.

Screen clipping taken: 04-06-2026 16:38

Each of these types of authorizations is called a **privilege**. We may authorize the user all, none, or a combination of these types of privileges on specified parts of a database, such as a relation or a view.

When a user submits a query or an update, the SQL implementation first checks if the query or update is authorized, based on the authorizations that the user has been granted. If the query or update is not authorized, it is rejected.

Screen clipping taken: 04-06-2026 16:41

4.6.1 Granting and Revoking of Privileges

The SQL standard includes the privileges **select**, **insert**, **update**, and **delete**. The privilege **all privileges** can be used as a short form for all the allowable privi-

Chapter 4 Intermediate SQL

leges. A user who creates a new relation is given all privileges on that relation automatically.

The SQL data-definition language includes commands to grant and revoke privileges. The **grant** statement is used to confer authorization. The basic form of this statement is:

```
grant <privilege list>  
on <relation name or view name>  
to <user/role list>;
```

Screen clipping taken: 04-06-2026 16:42

The *privilege list* allows the granting of several privileges in one command. The notion of roles is covered later, in Section 4.6.2.

The **select** authorization on a relation is required to read tuples in the relation. The following **grant** statement grants database users Amit and Satoshi **select** authorization on the *department* relation:

```
grant select on department to Amit, Satoshi;
```

Screen clipping taken: 04-06-2026 16:43

The update authorization on a relation allows a user to update any tuple in the relation. The update authorization may be given either on all attributes of the relation or on only some.

This **grant** statement gives users Amit and Satoshi update authorization on the *budget* attribute of the *department* relation:

```
grant update (budget) on department to Amit, Satoshi;
```

Screen clipping taken: 04-06-2026 16:46

The **insert** authorization on a relation allows a user to insert tuples into the relation. The **insert** privilege may also specify a list of attributes; any inserts to the relation must specify only these attributes, and the system either gives each of the remaining attributes default values (if a default is defined for the attribute) or sets them to *null*.

The **delete** authorization on a relation allows a user to delete tuples from a relation.

Screen clipping taken: 04-06-2026 16:46

To revoke an authorization, we use the **revoke** statement. It takes a form almost identical to that of **grant**:

```
revoke <privilege list>  
on <relation name or view name>  
from <user/role list>;
```

Thus, to revoke the privileges that we granted previously, we write

```
revoke select on department from Amit, Satoshi;  
revoke update (budget) on department from Amit, Satoshi;
```

Revocation of privileges is more complex if the user from whom the privilege is revoked has granted the privilege to another user. We return to this issue in Section 4.6.5.

Screen clipping taken: 04-06-2026 16:47